CSA0908 Java

CO1 - Assignment 1

Campus Smart Parking System

Name	Mohammed Rayan Shaikh
Register number	192424059
GitHub repository	https://github.com/mohdrayan01/CSA0908-JAVA
Submission date	20/10/2026

Introduction

The campus parking system must assign compatible free slots, create unique tickets, record entry and exit, calculate parking and charging fees, and apply valid passes. The proposed design uses encapsulated classes, a jagged floor-slot structure, polymorphic vehicle behaviour, interfaces for optional capabilities, and pricing rules that can evolve with the requirements. The updated design retains the core model while introducing time-aware pricing and pass types.

Task 1 - Understand the Problem and Identify the Objects

Inputs and outputs

Inputs include vehicle registration and category, pass information, entry and exit timestamps, facility floor capacities, and energy consumed for charging. Outputs include a floor/slot allocation, unique ticket, parking and charging fee, release success or failure, and clear errors for full facilities or invalid input.

Candidate classes and responsibilities

Candidate	Responsibility
ParkingFacility	Own floors and slots; find and reserve a compatible free slot; release slots.
ParkingSlot	Hold floor/slot identity, supported size or category, and optional parked vehicle.
Vehicle (abstract)	Hold common registration identity and expose shared fee operations.
TwoWheeler, Car, ElectricCar	Represent category-specific rates/capabilities.
ParkingTicket	Record unique number, vehicle, assigned slot, entry and exit timestamps.
ParkingService	Coordinate arrival, active-ticket checks, allocation, exit, fee and release.
ParkingPass	Represent pass type, owner/vehicle eligibility and validity period.

Chargeable	Expose charging-fee behaviour for vehicles with charging capability.
PricingRule	Calculate a fee from category, duration and applicable time rules.

Business rules and assumptions

- A slot holds at most one vehicle and only a compatible vehicle may use it.
- An active registration number has at most one ticket; ticket numbers are unique and never reused.
- Invalid release requests do not alter the facility. Negative durations and fees are rejected.
- Every started hour is charged as a whole hour in the initial policy; zero minutes costs zero.
- Valid monthly passes cover eligible students and staff; visitors pay unless a valid day pass applies.
- Charging is billed separately. Timestamps use LocalDateTime and the facility uses one configured local time zone.
- Assume registration numbers are normalized to uppercase and trimmed before comparison.

Nouns suggest domain objects such as Vehicle, Slot, Ticket and Pass; verbs suggest operations such as allocate, release, enter, exit, validate and calculateFee. Each class has a distinct responsibility so that a pricing change does not require rewriting slot management.

Alternative concept and complexity

Searching Vehicle objects by registration is O(n) for n vehicles unless an index such as HashMap is used. Adding an attribute adds a field and related methods in the class hierarchy. Parallel arrays regNo[], type[] and entryTime[] also require O(n) search, but all arrays must remain aligned by index. Both use O(n) record storage; object modeling costs object/reference overhead but keeps each record cohesive. Adding an attribute to parallel arrays requires another array and coordinated updates everywhere.

Task 2 - Choose the Data Representation

Value	Representation	Reason
Floor/slot index	int	Whole-number array indexes.
Registration	String	Alphanumeric identifier; normalize for comparisons.
Entry/exit	LocalDateTime	Date and time operations without hand-parsing.
Duration	long minutes	Avoids truncating fractional hours.
Money	BigDecimal	Predictable decimal currency arithmetic.
Energy	BigDecimal	Exact decimal charging calculation.

Represent floors as ParkingSlot[][] slots. Each row is a floor and may have a different length, so the structure is jagged. Access to a known floor and slot is O(1); searching all slots is O(S), where S is the total slot count.

Integer division truncates: 130 / 60 evaluates to 2, incorrectly billing two hours when each started hour is charged. For a positive duration, use (minutes + 59) / 60; validate that minutes are nonnegative first. Use long arithmetic if large values are possible. The calculation avoids converting minutes to an integer number of hours before rounding.

Alternative concept and complexity

A one-dimensional array with index = floor * slotsPerFloor + slot is convenient only when all floors have the same capacity. With different capacities, store a prefix offset per floor or use a jagged array. An ArrayList<ParkingSlot[]> also supports variable floor sizes. Direct lookup is O(1) with floor and slot indexes; free-slot scanning remains O(S). Each representation requires O(S) slot storage, with offsets requiring O(F) extra space for F floors.

Task 3 - Design the Control Logic

Allocation pseudocode

```
PROCEDURE allocate(vehicle)
  IF vehicle is null OR registration is invalid
    RETURN INVALID_INPUT
  END IF
  FOR floor <- 0 TO numberOfFloors - 1
    FOR slot <- 0 TO slots[floor].length - 1
      IF slots[floor][slot] is free AND compatible(vehicle, slots[floor][slot])
        reserve slots[floor][slot] for vehicle
        RETURN (floor, slot)
      END IF
    END FOR
  END FOR
  RETURN NO_SUITABLE_SLOT
END PROCEDURE
```

Fee pseudocode

```
FUNCTION calculateFee(vehicle, entry, exit, pass, energy)
  IF entry is null OR exit is null OR exit precedes entry
    RETURN INVALID_TIME
  END IF
  IF valid pass covers this vehicle and parking date
    parkingFee <- 0
  ELSE
    minutes <- elapsed whole/partial minutes according to stated policy
    hours <- 0 if minutes = 0, otherwise ceiling(minutes / 60)
    parkingFee <- apply pricing rule to vehicle and time interval
```

```
        IF daily maximum applies
            parkingFee <- minimum(parkingFee, daily maximum)
        END IF
    END IF
    chargingFee <- 0
    IF vehicle implements Chargeable
        chargingFee <- vehicle charging rate * energy
    END IF
    RETURN parkingFee + chargingFee
END FUNCTION
```

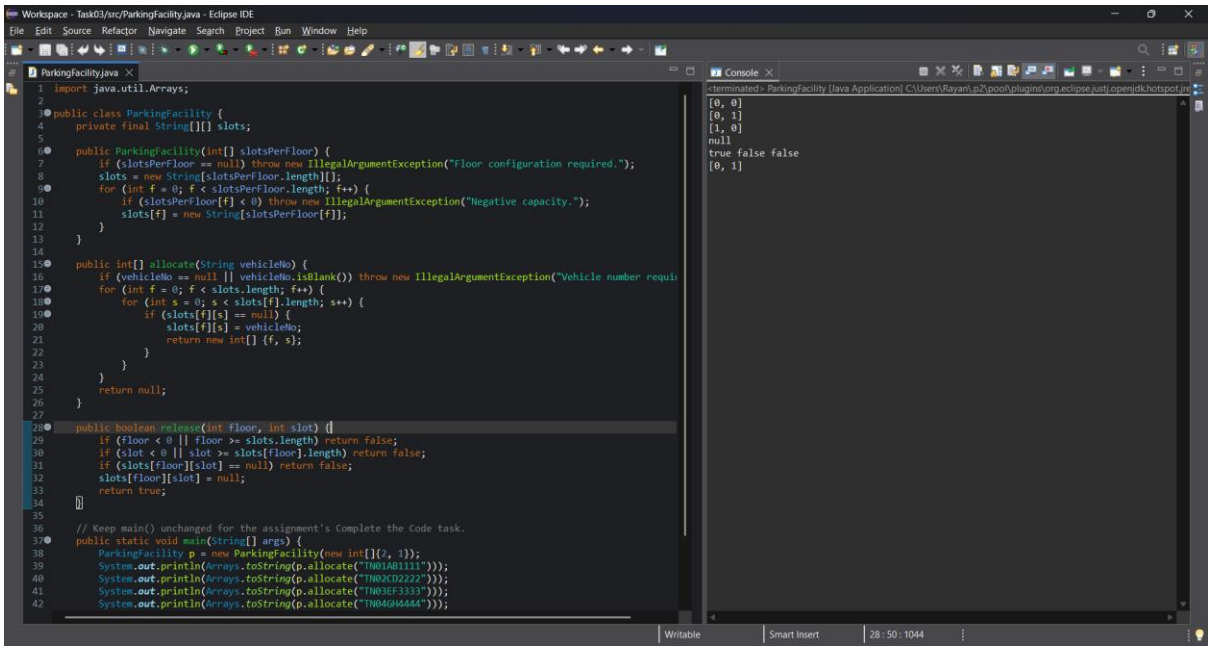
Completed supplied ParkingFacility TODOs

```
public int[] allocate(String vehicleNo) {
    for (int f = 0; f < slots.length; f++) {
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[] { f, s };
            }
        }
    }
    return null;
}

public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;
    slots[floor][slot] = null;
    return true;
}
```

Expected supplied-program output: [0, 0], [0, 1], [1, 0], null, true false false, [0, 1]. The assignment requires keeping its main() unchanged.

ParkingFacility execution output:



Alternative concept and complexity

A nested scan makes allocate O(S) worst-case and release O(1) when coordinates are given (the supplied implementation validates indexes in constant time). A stack of free-slot coordinates per compatibility group can make allocation and release O(1) average, using O(S) extra references. Per-floor free counts can skip full floors but do not alone identify a free slot. A robust implementation must keep counts/stacks consistent with actual occupancy.

Task 4 - Design Classes, Objects and Constructors

Vehicle should use a protected parameterized constructor that validates and stores registration. Subclasses call super(regNo). ParkingTicket should have a parameterized constructor requiring a ticket number, vehicle, slot and entry time so it cannot exist in a partially initialized state. Constructor chaining centralizes validation; this(...) can route overloads through one canonical constructor.

A copy constructor is not suitable for duplicate ticket issuance because it could duplicate a unique ticket number. Reissue should create a new ticket number and preserve an audit relationship if needed. A local reference is used to access an object allocated on the heap; methods pass and return object references.

Alternative concept and complexity

A default constructor followed by setters allows partially valid objects and spreads invariants across calls. Static factories can validate and return a fully initialized ticket, with named methods for different creation scenarios. Creating n tickets costs O(n) total time and O(n) retained space in all approaches. Constructors and factories do not change asymptotic cost, but they improve validity guarantees.

Task 5 - Apply Static Members and Object Life Cycle

If ticket numbers are globally unique across all facilities, a static long counter is appropriate. If numbering is independent per facility, keep the counter as a ParkingFacility instance field. Static methods are appropriate for operations not dependent on one instance, such as registration normalization. Static blocks are only needed for class-wide initialization.

```
private static long nextTicketNumber = 1;

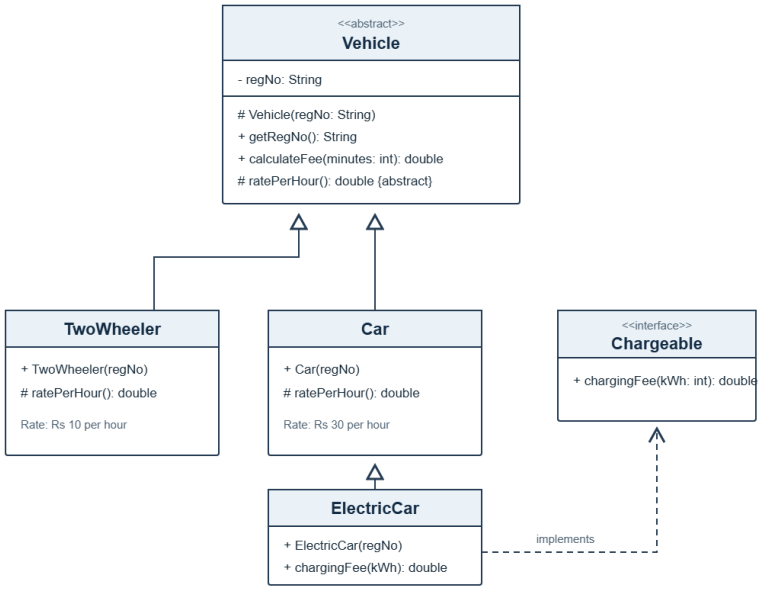
private static synchronized long issueTicketNumber() {
    return nextTicketNumber++;
}
```

The synchronization matters only if multiple threads can issue tickets concurrently. After exit, the active ticket map and slot should drop references to the current ticket and vehicle. If no other references remain, the ticket becomes eligible for garbage collection; historical records require a separate archive.

Alternative concept and complexity

A static counter uses O(1) storage for any number of facility objects and yields a shared sequence. An instance counter uses O(k) storage for k facilities and allows each facility to begin its own sequence. Both use O(n) total operations to generate n ticket IDs; ticket identity policy determines which is correct.

Task 6 - Design the Inheritance Hierarchy



This is hierarchical inheritance, with ElectricCar also showing multilevel inheritance through Car. Vehicle owns registration identity and common duration handling. Concrete vehicle classes provide category behaviour. super(regNo) invokes parent initialization. When

ElectricCar is created, Java runs Object, Vehicle, Car, then ElectricCar constructors (including each constructor body after its parent completes).

UML summary: Vehicle is abstract with private regNo, public getRegNo(), public calculateFee(...), and protected abstract rate/pricing behavior. TwoWheeler, Car, ElectricCar and DeliveryVan extend Vehicle. ElectricCar implements Chargeable. ParkingFacility contains ParkingSlot[][]; ParkingService uses ParkingFacility, PricingRule and a ticket repository.

Alternative concept and complexity

Composition stores a VehicleType or PricingRule inside Vehicle. Adding a category then usually requires a new rule/configuration rather than a subclass. Inheritance makes category identity explicit but may create many subclasses when combinations grow. Both calculate a fee in O(1) for a fixed rule; time-window integration is O(b) for b pricing intervals crossed. Composition adds a rule reference, O(1) space per vehicle.

Task 7 - Override and Specialise Behaviour

Subclasses can override ratePerHour() or a pricing strategy method. Use @Override so the compiler detects signature errors. Overrides must preserve a compatible signature and return type and cannot reduce visibility. Mark immutable registration and ticket identifiers final. Mark a class or method final only when extension is intentionally prohibited.

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Vehicle)) return false;
    Vehicle v = (Vehicle) other;
    return regNo.equals(v.regNo);
}

@Override
public int hashCode() {
    return regNo.hashCode();
}

@Override
public String toString() {
    return getClass().getSimpleName() + "[regNo=" + regNo + "]";
}
```

Equality by registration makes objects for the same vehicle equal regardless of subtype. This is useful for duplicate-registration prevention, but equals and hashCode must use the same normalized value. A switch-based calculator can be O(1) with fixed categories but requires modifying a central method and risks missed cases. Overriding keeps behavior in category classes; a new type can be added without editing existing ones when no shared policy changes.

Task 8 - Apply Polymorphism

Compile-time polymorphism can be demonstrated by overloads calculateFee(int minutes) and calculateFee(LocalDateTime entry, LocalDateTime exit). The compiler selects based on argument types. Runtime polymorphism uses a Vehicle reference to a Car object; overridden behavior is selected from the runtime object.

```
public double calculateFee(int minutes) {
    if (minutes < 0) throw new IllegalArgumentException("negative duration");
    int hours = (minutes == 0) ? 0 : (minutes + 59) / 60;
    return hours * ratePerHour();
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}

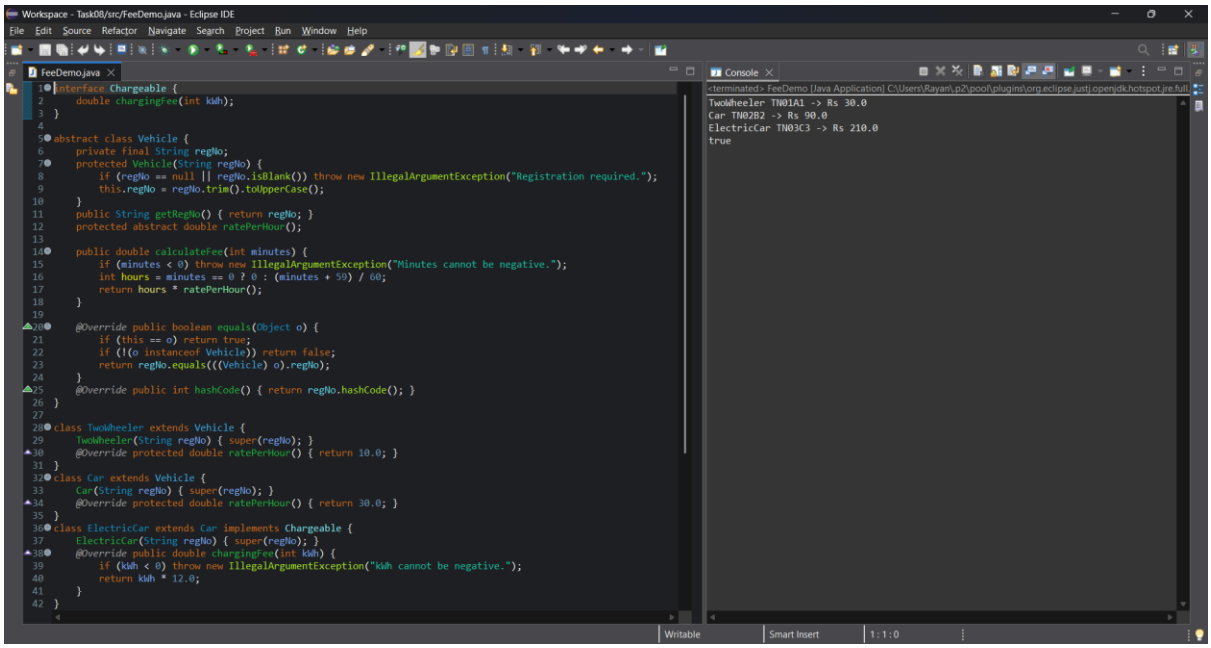
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("negative kWh");
        return kWh * 12;
    }
}
public BigDecimal calculateFee(LocalDateTime entry, LocalDateTime exit) {
    if (exit.isBefore(entry)) {
        throw new IllegalArgumentException("Exit precedes entry.");
    }

    long minutes = Duration.between(entry, exit).toMinutes();
    long hours = minutes == 0 ? 0 : (minutes + 59) / 60;

    return ratePerHour().multiply(BigDecimal.valueOf(hours));
}
```

For the supplied FeeDemo, also implement the Car constructor and ratePerHour() as above; ElectricCar chargingFee returns 12 * kWh. The expected output is TwoWheeler TN01A1 -> Rs 30.0; Car TN02B2 -> Rs 90.0; ElectricCar TN03C3 -> Rs 210.0; true. The last true occurs because the supplied equals compares registration numbers only. Whether that is suitable depends on whether registration identifies the same vehicle across subclasses; duplicate active registration must still be rejected.

FeeDemo execution output:



Alternative concept and complexity

With instanceof and casts, a method may perform up to k checks for k categories, O(k) worst-case, and central code grows as categories are added. Dynamic dispatch is effectively O(1) per method call and keeps behavior in classes. Both still have O(1) fee arithmetic for a fixed calculation.

Task 9 - Introduce Abstraction

Vehicle is abstract because it holds common state and behavior but should not be instantiated without a concrete category. Chargeable is an interface because charging is an optional capability that may be added to different classes. A class can implement multiple interfaces, supporting multiple capabilities without multiple class inheritance.

```
public interface Chargeable {
    BigDecimal chargingFee(BigDecimal kWh);
}

public abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
        if (regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("registration required");
        this.regNo = regNo.trim().toUpperCase(Locale.ROOT);
    }
    public String getRegNo() { return regNo; }
    public abstract BigDecimal calculateFee(
        LocalDateTime entry, LocalDateTime exit);
}
```

A default interface method suits optional shared behavior; a static interface method suits a related utility. Replacing Vehicle with an interface removes the normal shared constructor and instance state implementation, though interfaces can declare constants and default methods. Replacing Chargeable with an abstract class prevents a class that already extends Car from also extending that charging class. Calls remain O(1) dispatch and the interface does not require an extra domain field per object.

Task 10 - Enforce Encapsulation through Packages and Access Modifiers

parking.model: Vehicle, Car, ParkingSlot, ParkingTicket, ParkingPass
parking.service: ParkingFacility, ParkingService, PricingRule
parking.app: Main
parking.exception: NoSlotAvailableException, InvalidTicketException

Use private fields by default, public methods for supported operations, protected members only for intended subclass extension, and package-private helpers for package internals. Prefer immutable fields and validated constructors. Getters expose state; setters should be limited and validate before updating. Reject blank registration, negative duration/energy, invalid slot indexes and invalid pass periods.

Wrapper types such as Integer are required by many generic collections. Autoboxing converts int to Integer. Comparing Integer references with == may compare identity; use equals() or unbox to int. Validation takes O(1) per field update. Public fields force every caller to reproduce validation, increasing the number of error-prone call sites.

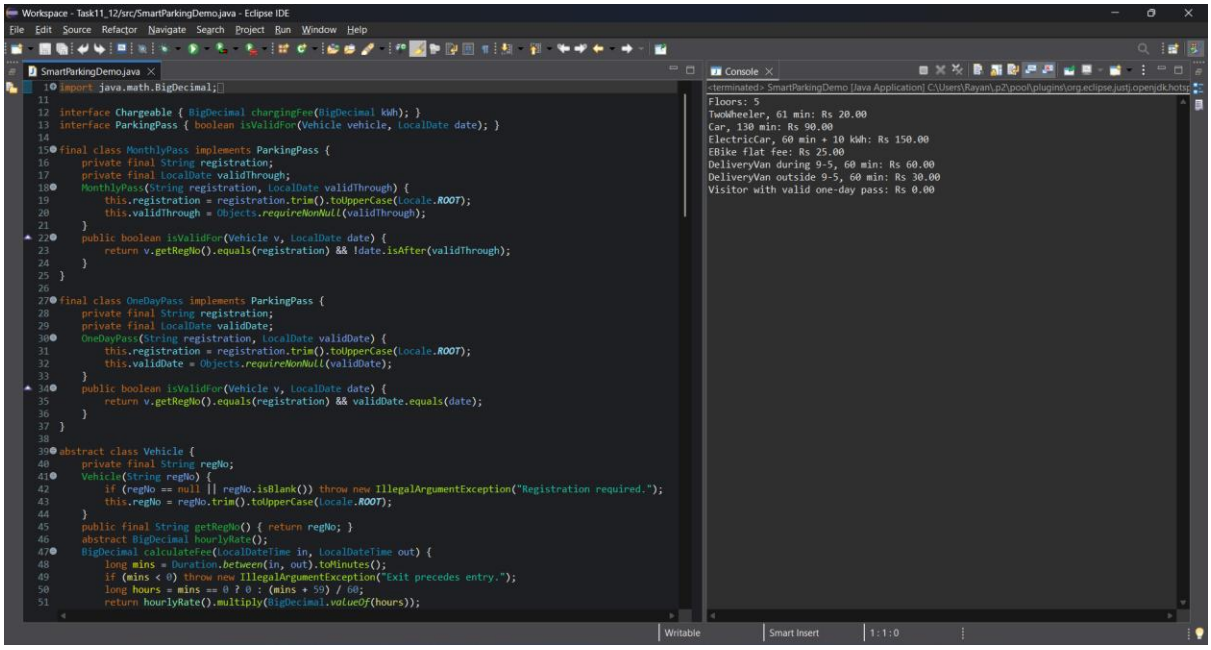
Task 11 - Implement, Test and Defend the Design

The service should reject duplicate active registrations, allocate only compatible slots, create a ticket after successful allocation, and on exit find the active ticket, calculate fees, release the slot and remove the active record. A HashMap<String, ParkingTicket> keyed by normalized registration is recommended for exit lookup.

Validation plan

Scenario	Expected behavior	Results
Different categories enter and exit	Tickets, category fees and release are correct	TwoWheeler: Rs 20.00 Car: Rs 90.00 ElectricCar including charging: Rs 150.00.
Last free slot / facility full	Last slot succeeds; next arrival gets no-slot result	Not tested yet.
Zero minutes / exactly 60 minutes	0 fee / one hour fee	Zero minutes: not tested. Electric car at 60 minutes: Rs 30.00 parking Rs 150.00 total with 10 kWh charging.
61 minutes / 130 minutes	2 / 3 started hours	Two-wheeler at 61 minutes: Rs 20.00

		car at 130 minutes: Rs 90.00.
Unknown category or duplicate registration	Reject input; preserve state	Not tested yet.
Invalid floor/slot or already-free slot	Release returns false; state unchanged	Not tested yet.
Monthly pass and EV charging	Eligible parking waived; charging separately billed	EV charging: Rs 150.00 total, including Rs 30.00 parking and Rs 120.00 charging. Monthly pass: not tested yet.



Hand calculation scenario

The demonstration run produced seven fees: two-wheeler Rs 20.00, car Rs 90.00, electric car including charging Rs 150.00, e-bike Rs 25.00, delivery van during peak time Rs 60.00, delivery van outside peak time Rs 30.00, and visitor with a valid one-day pass Rs 0.00. The hand-calculated total is Rs 20 + Rs 90 + Rs 150 + Rs 25 + Rs 60 + Rs 30 + Rs 0 = Rs 375.00. This agrees with the sum of the displayed program results.

Task 12 - Re-evaluate and Adapt the Design

A. Impact analysis

New categories are e-bikes and delivery vans. Delivery vans pay double the car rate from 9:00 AM through 5:00 PM. Visitors can buy a one-day pass. The facility now has five floors with varying capacities. Vehicle pricing, pass validation, and facility configuration are affected. A jagged array already supports floors with unequal slot counts.

B. Evaluation of the original design

The hierarchy can add subclasses for new categories, but time-dependent pricing should not be embedded as duplicated conditionals across multiple classes. Keep the pricing policy in a PricingRule abstraction. Existing vehicle and slot classes should remain unchanged where possible; identify in your implementation exactly which files needed modification.

C. Adaptation

Modify the design by retaining Vehicle, ParkingFacility and jagged floor storage, while introducing pricing strategies and explicit pass types. E-bike uses a flat-fee rule. DeliveryVan uses a time-window rule. OneDayPass stores its validity date and visitor identity. A vehicle can hold a PricingRule reference, or the service can select rules by category.

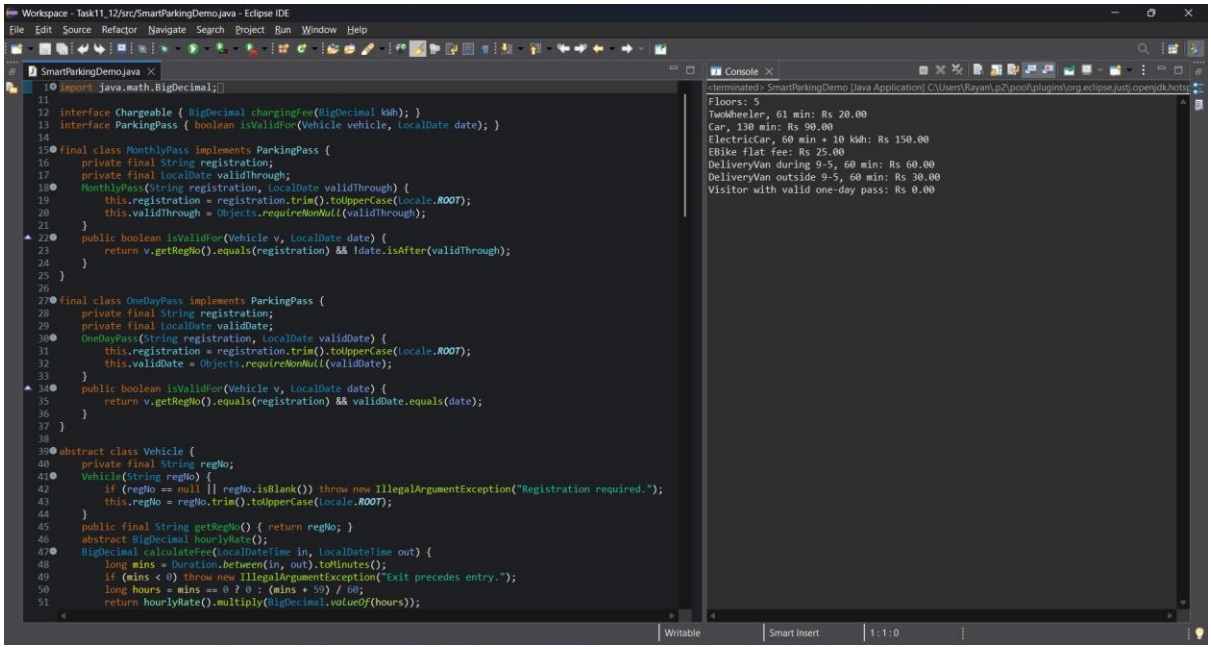
```
public interface PricingRule {
   BigDecimal calculateFee(Vehicle vehicle,
                           LocalDateTime entry,
                           LocalDateTime exit);
}
public final class FlatFeeRule implements PricingRule {
   private final BigDecimal fee;
   public FlatFeeRule(BigDecimal fee) { this.fee = fee; }
   public BigDecimal calculateFee(Vehicle v, LocalDateTime in,
                                  LocalDateTime out) {
      return fee;
   }
}
```

DeliveryVanRule should divide the stay into intervals at 9:00 AM and 5:00 PM boundaries. For each interval, apply twice the car rate when the interval lies inside the window and the normal delivery policy outside it. Then apply the assignment rounding and daily cap consistently. State explicitly whether the endpoint is inclusive and how partial hours crossing a boundary are rounded; these details are assumptions because the assignment does not define them.

OneDayPass should hold a LocalDate validDate and owner/registration identity. It is valid only for the matching date and eligible visitor, and should be rejected after expiry. The requirement does not state the pass price; make it a named configurable constant or mark it as an assumption and get faculty clarification.

D. Evidence-based defence

Recommendation: modify the design. The jagged array and common Vehicle model already meet the structural requirements. PricingRule isolates time-dependent and flat fees; pass objects encode validity.



- For this demo, I added EBike, DeliveryVan, OneDayPass and MonthlyPass, and used a five-floor ParkingFacility.
- E-bike test: the program charged a flat Rs 25.00. Actual output: Rs 25.00.
- Delivery van test: 60 minutes starting at 10:00 AM cost Rs 60.00; 60 minutes starting at 6:00 PM cost Rs 30.00. Both matched the program′s peak and off-peak rates.
- Valid one-day pass test: the visitor′s parking fee was Rs 0.00. An expired one-day pass was not tested in this run.

A suitable defence is that new pricing is implemented as a new rule rather than edits to slot allocation. Confirm this claim against your actual code diff and tests before submission.

Conclusion

The design models vehicles, slots, tickets, passes and pricing as separate responsibilities. Jagged arrays support variable floor capacity, while inheritance and interfaces represent common identity and optional capabilities. A pricing strategy abstraction supports the changed time-window and flat-fee rules. The final report must include executed outputs, actual test results, screenshots, the working GitHub link, progressive commits and the student's own explanation of the design.